

1. 遞迴

「遞迴是小事化小，要記得小事化無。」

「暴力或許可以解決一切，但是可能必須等到地球毀滅。」

「遞迴搜尋如同末日世界，心中有樹而程式(城市)中無樹。」

本章介紹遞迴函數的基本方法與運用，也介紹使用窮舉暴搜的方法。

1.1. 基本觀念與用法

遞迴在數學上是函數直接或間接以自己定義自己，在程式上則是函數直接或間接呼叫自己。遞迴是一個非常重要的程式結構，在演算法上扮演重要的角色，許多的算法策略都是以遞迴為基礎出發的，例如分治與動態規劃。學習遞迴是一件重要的事，不過遞迴的思考方式與一般的不同，它不是直接的給予答案，而是間接的說明答案與答案的關係，不少程式的學習者對遞迴感到困難。以簡單的階乘為例，如果採直接的定義：

對正整數 n ， $\text{fac}(n)$ 定義為所有不大於 n 的正整數的乘積，也就是

$$\text{fac}(n) = 1 \times 2 \times 3 \times \dots \times n。$$

如果採遞迴的定義，則是：

$$\text{fac}(1) = 1; \text{ and}$$

$$\text{fac}(n) = n \times \text{fac}(n-1) \text{ for } n > 1。$$

若以 $n=3$ 為例，直接的定義告訴你如何計算 $\text{fac}(3)$ ，而遞迴的定義並不直接告訴你 $\text{fac}(3)$ 是多少，而是

$$\text{fac}(3) = 3 \times \text{fac}(2)，\text{而}$$

$$\text{fac}(2) = 2 \times \text{fac}(1)，\text{最後才知道}$$

$$\text{fac}(1) = 1。$$

所以要得到 $\text{fac}(3)$ 的值，我們再逐步迭代回去，

$$\text{fac}(2) = 2 \times \text{fac}(1) = 2 \times 1 = 2，$$

$$\text{fac}(3) = 3 \times \text{fac}(2) = 3 \times 2 = 6。$$

現在大多數的程式語言都支援遞迴函數的寫法，而函數呼叫與轉移的過程，正如上面逐步推導的過程一樣，雖然過程有點複雜，但不是寫程式的人的事，以程式來撰寫遞迴函數幾乎就跟定義一模一樣。上面的階乘函數以程式來實作可以寫成：

```
int fac(int n) {
    if (n == 1) return 1;
    return n * fac(n-1);
}
```

再舉一個很有名的費式數列 (Fibonacci) 來作為例子。費式數列的定義：

$$F(1) = F(2) = 1;$$

$$F(n) = F(n-1) + F(n-2) \text{ for } n > 2.$$

以程式來實作可以寫成：

```
int f(int n) {
    if (n <= 2) return 1;
    return f(n-1) + f(n-2);
}
```

遞迴函數一定會有終端條件 (也稱邊界條件)，例如上面費式數列的 $F(1) = F(2) = 1$ 以及階乘的 $\text{fac}(1) = 1$ ，通常的寫法都是**先寫終端條件**。遞迴程式與其數學定義非常相似，通常只要把數學符號換成適當的指令就可以了。上面的兩個例子中，程式裡面就只有一個簡單的計算，有時候也會帶有迴圈，例如 Catalan number 的遞迴式定義：

$$C_0 = 1 \text{ and } C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i} \text{ for } n \geq 1.$$

程式如下：

```
int cat(int n) {
    if (n == 0) return 1;
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += cat(i) * cat(n-1-i);
    return sum;
}
```

遞迴程式雖然好寫，但往往有效率不佳的問題，通常遞迴函數裡面只呼叫一次自己的效率比較不會有問題，但如果像 Fibonacci 與 Catalan number，一個呼叫兩個或是更多個，其複雜度通常都是指數成長，演算法裡面有些策略是來改善純遞迴的效率，例如動態規劃，這在以後的章節中再說明。

遞迴通常使用的時機有兩類：

- 根據定義來實作。
- 為了計算答案，以遞迴來進行窮舉暴搜。

以下我們分別來舉一些例題與習題。

1.2. 實作遞迴定義

P-1-1. 合成函數(1)

令 $f(x)=2x-1$, $g(x,y)=x+2y-3$ 。本題要計算一個合成函數的值，例如 $f(g(f(1),3))=f(g(1,3))=f(4)=7$ 。

Time limit: 1 秒

輸入格式：輸入一行，長度不超過 1000，它是一個 f 與 g 的合成函數，但所有的括弧與逗號都換成空白。輸入的整數絕對值皆不超過 1000。

輸出：輸出函數值。最後答案與運算過程不會超過正負 10 億的區間。

範例輸入：
 $f \ g \ f \ 1 \ 3$
 範例輸出：
 7

讀入 f 呼叫下一個函數需要 1 個參數，讀入 g 需要 2 個參數呼叫下一個函數，讀入 f 需要一個參數，讀入 1 回傳給 f ，計算 $2 \times 1 - 1 = 1$ ，完成 g 的第一個參數，讀入 3，回傳給 g 計算 $2 \times 1 + 2 \times 3 - 3 = 4$ ，將 4 傳回給 f ，計算 $2 \times 4 - 1 = 7$ ，即為答案

題解：合成函數的意思是它的傳入參數可能是個數字也可能是另外一個函數值。以遞迴的觀念來思考，我們可以將一個合成函數的表示式定義為一個函式 `eval()`，這個函式從輸入讀取字串，回傳函數值。其流程只有兩個主要步驟，第一步是讀取一個字串，根據題目定義，這個字串只有 3 種情形： f ， g 或是一個數字。第二步是根據這個字串分別去進行 f 或 g 函數值的計算或是直接回傳字串代表的數字。至於如何計算 f 與 g 的函數值呢？如果是 f ，因為它有一個傳入參數，這個參數也是個合成函數，所以我們遞迴呼叫 `eval()` 來取得此參數值，再根據定義計算。如果是 g ，就要呼叫 `eval()` 兩次取得兩個參數。以下是演算法流程：

```
int eval() // 一個遞迴函式，回傳表示式的值
    讀入一個空白間隔的字串 token;
    if token 是 f then
```

```

    x = eval();
    return 2*x - 1;
else if token 是 g then
    x = eval();
    y = eval();
    return x + 2*y - 3;
else // token 是一個數字字串
    return token 代表的數字
end of eval()

```

程式實作時，每次我們用字串的輸入來取得下一個字串，而字串可能需要轉成數字，這可以用庫存函數 `atoi()` 來做。

Tip 自動跳過空白

```

// p 1.1a
#include <bits/stdc++.h>
int eval(){
    int val, x, y, z;
    char token[7]; // 題目說input絕對值不大於1000 ∴ -1000 ≤ input ≤ 1000
    scanf("%s", token);
    if (token[0] == 'f') {
        x = eval();
        return 2*x - 1;
    } else if (token[0] == 'g') {
        x = eval();
        y = eval();
        return x + 2*y - 3;
    } else {
        return atoi(token); // 終端條件
    }
}

int main() {
    printf("%d\n", eval());
    return 0;
}

```

7 極端情況

eg. `atoi` ↓ -1000

終端條件

`atoi()` 是一個常用的函數，可以把字串轉成對應的整數，名字的由來是 `ascii-to-int`。當然也有其它的方式來轉換，這一題甚至可以只用 `scanf()` 就可以，這要利用 `scanf()` 的回傳值。我們可以將 `eval()` 改寫如下，請看程式中的註解。

```

// p_1_1b
int eval(){
    int val, x, y, z;
    char c;
    // first try to read an int, if successful, return the int
    if (scanf("%d",&val) == 1) {
        return val;
    }
    // otherwise, it is a function name: f or g
    scanf("%c", &c);
    if (c == 'f') {

```

5 9 5 1 3

```

    x = eval(); // f has one variable
    return 2*x-1;
} else if (c == 'g') {
    x = eval(); // g has two variables
    y = eval();
    return x + 2*y -3;
}
}

```

下面是個類似的習題。

Q-1-2. 合成函數(2) (APCS201902)

令 $f(x)=2x-3$; $g(x,y)=2x+y-7$; $h(x,y,z)=3x-2y+z$ 。本題要計算一個合成函數的值，例如 $h(f(5),g(3,4),3)=h(7,3,3)=18$ 。

Time limit: 1 秒

輸入格式：輸入一行，長度不超過 1000，它是一個 f , g , 與 h 的合成函數，但所有的括弧與逗號都換成空白。輸入的整數絕對值皆不超過 1000。

輸出：輸出函數值。最後答案與運算過程不會超過正負 10 億的區間。

範例輸入：

h f 5 g 3 4 3

範例輸出：

18

每個例題與習題都若干筆測資在檔案內，請自行練習。再看下一個例題。

P-1-3. 棍子中點切割

有一台切割棍子的機器，每次將一段棍子會送入此台機器時，機器會偵測棍子上標示的可切割點，然後計算出最接近中點的切割點，並於此切割點將棍子切割成兩段，切割後的每一段棍子都會被繼續送入機器進行切割，直到每一段棍子都沒有切割點為止。請注意，如果最接近中點的切割點有二，則會選擇座標較小的切割點。每一段棍子的切割成本是該段棍子的長度，輸入一根長度 L 的棍子上面 N 個切割點位置的座標，請計算出切割總成本。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 N 與 L 。第二行有 N 個正整數，依序代表由小到大的切割點座標 $p[1] \sim p[N]$ ，數字間以空白隔開，座標的標示的方式是以棍子左端為 0，而右端為 L 。 $N \leq 5e4$ ， $L < 1e9$ 。

輸出：切割總成本點。

範例輸入：

4 10

1 2 4 6

1 2 3 4 5

範例輸出：

22

範例說明：第一次會切割在座標 4 的位置，切成兩段 $[0, 4]$ ， $[4, 10]$ ，成本 10；

$[0, 4]$ 切成 $[0, 2]$ 與 $[2, 4]$ ，成本 4；

$[4, 10]$ 切成 $[4, 6]$ 與 $[6, 10]$ ，成本 6；

$[0, 2]$ 切成 $[0, 1]$ 與 $[1, 2]$ ；成本 2；

總成本 $10+4+6+2 = 22$

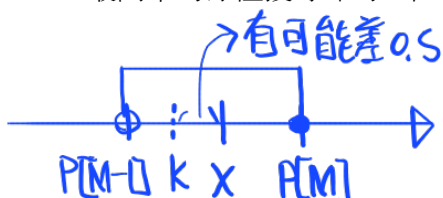
P-1-3 題解：棍子切斷後，要針對切開的兩段重複做切割的動作，所以是遞迴的題型。因為切點的座標值並不會改變，我們可以將座標的陣列放在全域變數中，遞迴函數需要傳入的是本次切割的左右端點。因為總成本可能大過一個 `int` 可以存的範圍，我們以 `long long` 型態來宣告變數，函數的架構很簡單：

```
// 座標存於 p[]
// 遞迴函式，回傳此段的總成本
long long cut(int left, int right) {
    找出離中點最近的切割點 m;
    return p[right]-p[left] + cut(left,m) + cut(m,right);
}
```

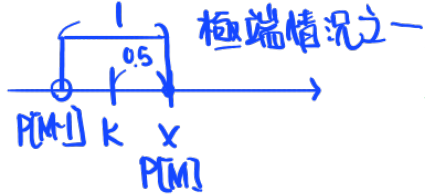
至於如何找到某一段的中點呢？離兩端等距的點座標應該是

$$x = (p[right] + p[left]) / 2$$

所以我們要找某個 m ，滿足 $p[m-1] < x \leq p[m]$ ，然後要找的切割點就是 $m-1$ 或 m ，看這兩點哪一點離中點較近，如相等就取 $m-1$ 。這一題因為數值的範圍所限，採取最簡單的線性搜尋即可，但二分搜是更快的方法，因為座標是遞增的。以下看實作，先



如果設定是 $p[m-1] < x \leq p[m]$ 的話， $p[m]$ 會在 x 的右邊或在 x 上，不然左邊的話這個區間就不包含 x 了！當然也可以設定 $p[m] \geq x > p[m+1]$ ，以此類推。



兩端點必分佈於中點 x 的兩側，或右端點恰位於 x 上。即便 k 因整數捨位而較 x 左移 0.5，只要端點間距最小為 1，該 0.5 的誤差仍不足以使切割點跳脫 x 的正確判定區間。

AP325 v:1.5 (20240921)

看以線性搜尋的範例程式。

小數無條件捨去
↑ when type 是 int

這裡有些細節要留意，當中點的位置非整數時， k 因為捨位誤差比真正的中點位置小 0.5，若 $p[m] == k$ ，它可能在真正中點的左方，但根據題意，此時要取的中點就是 $p[m]$ 。此外，在比較 $m-1$ 以及 m 與中點的距離時，這裡的寫法是比较與端點的距離。與端點較遠的就是離中點較近。當然也有其他的寫法。

```
// p_1_3a, linear search middle-point
#include <stdio>
#define N 50010
typedef long long LL;
LL p[N];

// find the cut in (left, right), and then recursively
LL cut(int left, int right) {
    if (right-left <= 1) return 0;
    LL len = p[right]-p[left], k = (p[right]+p[left])/2;
    int m = left;
    while (p[m] < k) m++; // linear search the first >= k
    if (p[m-1]-p[left] >= p[right]-p[m]) // check if m-1 is better
        m--;
    return len + cut(left, m) + cut(m, right);
}

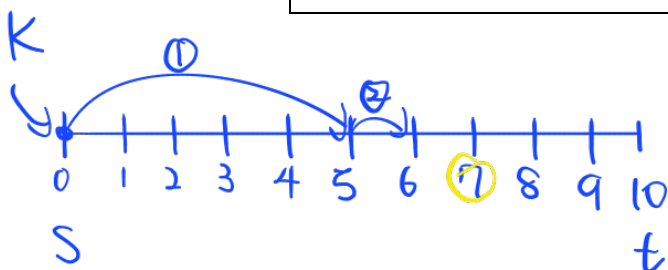
int main() {
    int i, n, l;
    scanf("%d%d", &n, &l);
    p[0]=0; p[n+1]=l; // left and right ends
    for (i=1; i<=n; i++) scanf("%lld", &p[i]);
    printf("%lld\n", cut(0, n+1));
    return 0;
}
```

$N \leq 5 \times 10^4$ (習慣多一點)

記得 right, left 是當 index 喔!

主程式中我們只需做輸入的動作，我們把頭尾加上左右端的座標，然後直接呼叫遞迴函數取得答案。如果採用二分搜來找中點，我們可以自己寫，也可以呼叫 C++ STL 的庫存函數。二分搜的寫法通常有兩種：一種(比較常見的)是維護搜尋範圍的左右端，每次以中點來進行比較，縮減一半的範圍。在陣列中以二分搜搜尋某個元素的話，這種方法是不會有甚麼問題，但是二分搜應用的範圍其實很廣，在某些場合這個方法很容易不小心寫錯。這裡推薦另外一種二分搜的寫法，它的寫法很直覺也不容易寫錯。

```
// 跳躍法二分搜
// 假設遞增值存於 p[]，在 p[s]~p[t] 找到最後一個 < x 的位置
k = s; // k 存目前位置，jump 是每次要往前跳的距離，逐步所減
for (jump = (t - s)/2; jump > 0; jump /= 2) {
    while (k+jump < t && p[k+jump] < x) // 還能往前跳就跳
        k += jump;
}
```



$x=7$
22

找到小於目標最近的位置，直到 $jump=0$ 被 int 捨去跳出 for 迴圈



唯一要提醒的有兩點：第一是要先確認 $p[s] < x$ ，否則最後的結果 $m=s$ 而 $p[m] \geq x$ ；第二是內迴圈要用 `while` 而不能只用 `if`，因為事實上內迴圈最多會做兩次。

我們來看看應用在這題時的寫法，以下只顯示副程式，其他部分沒變就省略了。

```
LL cut(int left, int right) {
    if (right-left<=1) return 0;
    int m=left;
    LL k=(p[right]+p[left])/2;
    for (int jump=(right-left)/2; jump>0; jump>>=1) {
        while (m+jump<right && p[m+jump]<k)
            m+=jump;
    }
    if (p[m]-p[left] < p[right]-p[m+1]) 記得控制誰要檢查另外一邊有沒有比較好
        m++;
    return p[right]-p[left] + cut(left, m) + cut(m, right);
}
```

我們也可以呼叫 C++ STL 中的函數來做二分搜。以下程式是一個範例。

呼叫 `lower_bound(s,t,x)` 會在 $[s,t)$ 的區間內找到第一個 $\geq x$ 的位置，回傳的是位置，所以把它減去起始位置就是得到索引值。

```
// 偷懶的方法，加以下這兩行就可以使用 STL 中的資料結構與演算法函數
#include <bits/stdc++.h>
using namespace std;
#define N 50010
typedef long long LL;
LL p[N];

// find the cut in (left,right), and then recursively
LL cut(int left, int right) {
    if (right-left<=1) return 0;
    LL k=(p[right]+p[left])/2;

    int m=lower_bound(p+left, p+right,k)-p;
    if (p[m-1]-p[left] >= p[right]-p[m])
        m--;
    return p[right]-p[left] + cut(left, m) + cut(m, right);
}
```

有關二分搜在下一章裡面會有更多的說明與應用。

Q-1-4. 支點切割 (APCS201802) (@@)

輸入一個大小為 N 的一維整數陣列 $p[]$ ，要找其中一個所謂的最佳切點將陣列切成左右兩塊，然後針對左右兩個子陣列繼續切割，切割的終止條件有兩個：子陣列範圍小於 3 或切到給定的層級 K 就不再切割。而所謂最佳切點的要求是讓左右各點數字與到切點距離的乘積總和差異盡可能的小，但不可以是兩端點，也就是說，若區段的範圍是 $[s, t]$ ，則要找出切點 $m \in [s+1, t-1]$ ，使得 $|\sum_{i=s}^t p[i] \times (i - m)|$ 越小越好，如果有兩個最佳切點，則選擇編號較小的。所謂切割層級的限制，第一次切以第一層計算。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 N 與 K 。第二行有 N 個正整數，代表陣列內容 $p[1] \sim p[N]$ ，數字間以空白隔開，總和不超過 10^9 。 $N \leq 50000$ ，切割層級限制 $K < 30$ 。

輸出：所有切點的 $p[]$ 值總和。

範例一輸入： <pre>7 3 2 4 1 3 7 6 9</pre>	範例一輸出： <pre>11</pre>
範例二輸入： <pre>5 1 1 2 3 4 100</pre>	範例二輸出： <pre>4</pre>

範例一說明：第一層切在 7，切成第二層的 $[2, 4, 1, 3]$ 與 $[6, 9]$ 。左邊 $[2, 4, 1, 3]$ 切在 4 與 1 都是最佳，選擇 4 來切，切成 $[2]$ 與 $[1, 3]$ ，右邊 $[6, 9]$ 不到 3 個就不切了。第三層都不到 3 個，所以終止。總計切兩個位置 $7+4=11$ 。

範例二說明：第一層切在 4 (注意切點不可在端點)，切出 $[1, 2, 3]$ 與 $[100]$ ，因為 $K=1$ ，所以不再切。

提示：與 P_1_3 類似，只是找切割點的定義不同，終端條件多了一個切割層級。

Q-1-5. 二維黑白影像編碼 (APCS201810)

假設 n 是 2 的冪次，也就是存在某個非負整數 k 使得 $n = 2^k$ 。將一個 $n \times n$ 的黑白影像以下列遞迴方式編碼：

$$k \geq 0, k \in \mathbb{Z}$$

如果每一格像素都是白色，我們用 0 來表示；

如果每一格像素都是黑色，我們用 1 來表示；

否則，並非每一格像素都同色，先將影像均等劃分為四個邊長為 $n/2$ 的小正方形後，然後表示如下：先寫下 2，之後依續接上左上、右上、左下、右下四塊的編碼。

輸入編碼字串 S 以及影像尺寸 n ，請計算原始影像中有多少個像素是 1。

Time limit: 1 秒

Tip: 可以簡化測資

輸入格式：第一行是影像的編碼 S ，字串長度小於 1,100,000。第二行為正整數 n ， $1 \leq n \leq 1024$ ，中 n 必為 2 的幕次。

輸出格式：輸出有多少個像素是 1。

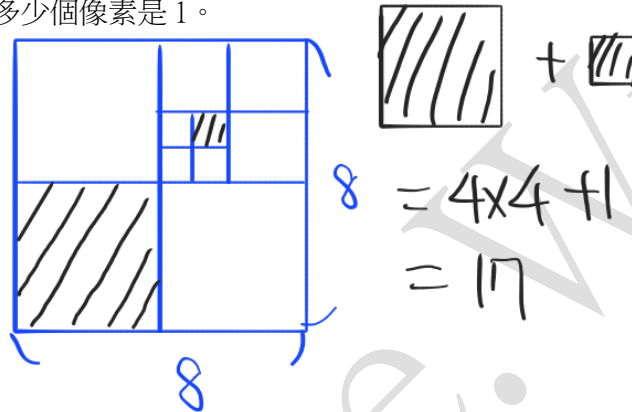
範例輸入：

2020020100010

8

範例輸出：

17



1.3. 以遞迴窮舉暴搜

窮舉 (Enumeration) 與暴搜 (Brute Force) 是一種透過嘗試所有可能來搜尋答案的演算法策略。通常它的效率很差，但是在有些場合也是沒有辦法中的辦法。暴搜通常是窮舉某種組合結構，例如： n 取 2 的組合，所有子集合，或是所有排列等等。因為暴搜也有效率的差異，所以還是有值得學習之處。通常以迴圈窮舉的效率不如以遞迴方式來做，遞迴的暴搜方式如同以樹狀圖來展開所有組合，所以也稱為分枝演算法或 Tree searching algorithm，這類演算法可以另外加上一些技巧來減少執行時間，不過這個部份比較困難，這裡不談。

以下舉一些例子，先看一個迴圈窮舉的例題，本題用來說明，未附測資。

P-1-6. 最接近的區間和

假設陣列 $A[1..n]$ 中存放著某些整數，另外給了一個整數 K ，請計算哪一個連續區段的和最接近 K 而不超過 K 。≤

(這個問題有更有效率的解，在此我們先說明窮舉的解法。)

要尋找的是一個連續區段，一個連續區段可以用一對駐標 $[i, j]$ 來定義，因此我們可以窮舉所有的 $1 \leq i \leq j \leq n$ 。剩下的問題是對於任一區段 $[i, j]$ ，如何計算 $A[i..j]$ 區間的和。最直接的做法是另外用一個迴圈來計算，這導致以下的程式：

```
// O(n^3) for range-sum
int best = K; // solution for empty range
for (int i=1; i<=n; i++) {
    for (int j=i; j<=n; j++) {
        int sum=0;
        for (int r=i; r<=j; r++)
            sum += A[r];
        if (sum<=K && K-sum<best)
            best = K-sum;
    }
}
printf("%d\n", best);
```

窮舉每個區間，分別算出每個區間和

$\because \text{sum} \leq K \therefore K - \text{sum} \geq 0$
找最接近的 sum

上述程式的複雜度是 $O(n^3)$ 。如果我們認真想一下，會發現事實上這個程式可以改進的。對於固定的左端 i ，若我們已經算出 $[i, j]$ 區間的和，那麼要計算 $[i, j+1]$ 的區間和只需要再加上 $A[j+1]$ 就可以了，而不需要整段重算。於是我們可以得到以下 $O(n^2)$ 的程式：

```
// O(n^2) for range-sum
int best = K; // solution for empty range
for (int i=1; i<=n; i++) {
    int sum=0;
    for (int j=i; j<=n; j++) {
        sum += A[j]; // sum of A[i] ~ A[j]
        if (sum<=K && K-sum<best)
            best = K-sum;
    }
}
printf("%d\n", best);
```

算出 $i=1, 2, 3, \dots$ 時的所有區間和，並利用前一次區間和，算出下一次區間和，不需要重頭算

另外一種 $O(n^2)$ 的程式解法是利用前綴和 (prefix sum)，前綴和是指：對每一項 i ，從最前面一直加到第 i 項的和，也就是定義 $ps[i] = \sum_{j=1}^i A[j]$ ，前綴和有許多應用，基本上，我們可以把它看成一個前處理，例如，如果已經算好所有的前綴和，那麼，對任意區間 $[i, j]$ ，我們只需要一個減法就可以計算出此區間的和，因為

$$\sum_{r=i}^j A[r] = ps[j] - ps[i-1]。$$

此外，我們只需要 $O(n)$ 的運算就可以計算出所有的前綴和，因為

$ps[i] = ps[i-1] + A[i]$ 。以下是利用 prefix-sum 的寫法，為了方便，我們設 $ps[0] = 0$ 。

```
// O(n^2) for range-sum, using prefix sum
ps[0]=0;
for (int i=1; i<=n; i++)
    ps[i]=ps[i-1]+A[i];
int best = K; // solution for empty range
for (int i=1; i<=n; i++) {
    for (int j=i; j<=n; j++) {
        int sum = ps[j] - ps[i-1];
        if (sum<=K && K-sum<best)
            best = K-sum;
    }
}
printf("%d\n", best);
```

$\lambda=1, j=1 \Rightarrow P[1]-0$
 $j=2 \Rightarrow P[2]-0$
 $j=3 \Rightarrow P[3]-0$
 $\lambda=2, j=2 \Rightarrow P[2]-P[1]$
 $j=3 \Rightarrow P[3]-P[1]$
 $\lambda=3, j=3 \Rightarrow P[3]-P[2]$
 $P[1] \quad P[2] \quad P[3]$

接下來看一個暴搜子集合的例題。

P-1-7. 子集合乘積

輸入 n 個正整數，請計算其中有多少組合的相乘積除以 P 的餘數為 1，每個數字可以選取或不選取但不可重複選，輸入的數字可能重複。 $P=10009$ ， $0 < n < 26$ 。

輸入第一行是 n ，第二行是 n 個以空白間隔的正整數。

輸出有多少種組合。若輸入為 $\{1, 1, 2\}$ ，則有三種組合，選第一個 1，選第 2 個 1，以及選兩個 1。

time limit = 1 sec。

我們以窮舉所有的子集合的方式來找答案，這裡的集合是指 multi-set，也就是允許相同元素，這只是題目描述上的問題，對解法沒有影響。要窮舉子集合有兩個方法：迴圈窮舉以及遞迴，遞迴會比較好寫也較有效率。先介紹迴圈窮舉的方法。

因為通常元素個數很有限，我們可以用一個整數來表達一個集合的子集合：第 i 個 bit 是 1 或 0 代表第 i 個元素在或不在此子集合中。看以下的範例程式：

```
// subset product = 1 mod P, using loop
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n, ans=0;
    long long P=10009, A[26];
    scanf("%d", &n);
    for (int i=0; i<n; i++) scanf("%lld", &A[i]);
    for (int s=1; s<=(1<<n); s++) { // for each subset s
        long long prod=1;
        for (int j=0; j<n; j++) { // check j-th bit
            if (s & (1<<j)) // if j-th bit is 1
```

Tip:

$$(A \times B) \bmod P = [(A \bmod P) \times (B \bmod P)] \bmod P$$

```

        prod = (prod*A[j])%P; // remember %
    }
    if (prod==1) ans++;
}
printf("%d\n", ans);
}

```

以下是遞迴的寫法。為了簡短方便，我們把變數都放在全域變數。遞迴副程式 `rec(i, prod)` 之參數的意義是指目前考慮第 i 個元素是否選取納入子集合，而 `prod` 是目前已納入子集合元素的乘積。

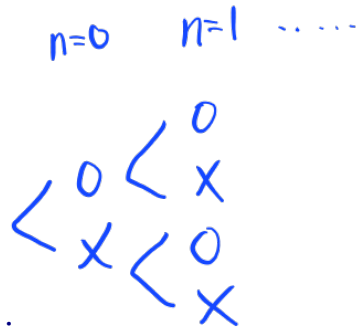
遞迴的寫法時間複雜度是 $O(2^n)$ 而迴圈的寫法時間複雜度是 $O(n \cdot 2^n)$ 。

```

// subset product = 1 mod P, using recursion
#include<bits/stdc++.h>
using namespace std;
int n, ans=0;
long long P=10009, A[26];
// for i-th element, current product=prod
void rec(int i, int prod) {
    if (i>=n) { // terminal condition
        if (prod==1) ans++;
        return;
    }
    rec(i+1, (prod*A[i])%P); // select A[i]
    rec(i+1, prod); // discard A[i]
    return;
}

int main() {
    scanf("%d", &n);
    for (int i=0; i<n; i++) scanf("%lld", &A[i]);
    ans=0;
    rec(0,1);
    printf("%d\n", ans-1); // -1 for empty subset
    return 0;
}

```



Q-1-8. 子集合的和 (APCS201810, subtask)

輸入 n 個正整數，請計算各種組合中，其和最接近 P 但不超過 P 的和是多少。每個元素可以選取或不選取但不可重複選，輸入的數字可能重複。 $P \leq 1000000009$, $0 < n < 26$ 。

Time limit: 1 秒

輸入格式：第一行是 n 與 P ，第二行是 n 個可挑選的正整數，大小不會超過 P ，同行數字以空白間隔。

輸出格式：最接近 P 但不超過 P 的和。

範例輸入：

```
5 17
5 5 8 3 10
```

範例輸出：

```
16
```

接著舉一個窮舉排列的例子。西洋棋的棋盤是一個 8×8 的方格，其中皇后的攻擊方式是皇后所在位置的八方位不限距離，也就是只要是在同行、同列或同對角線 (包含 45° 與 135° 度兩條對斜線)，都可以攻擊。一個有名的八皇后問題是問說：在西洋棋盤上有多少種擺放方式可以擺上 8 個皇后使得彼此之間都不會被攻擊到。這個問題可以延伸到不一定限於是 8×8 的棋盤，而是 $N \times N$ 的棋盤上擺放 N 個皇后。八皇后問題有兩個版本，這裡假設不可以旋轉棋盤。

P-1-9. N-Queen 解的個數

計算 N-Queen 有幾組不同的解，對所有 $0 < N < 12$ 。

(本題無須測資)

要擺上 N 個皇后，那麼顯然每一列恰有一個皇后，不可以多也不可以少。所以每一組解需要表示每一列的皇后放置在哪一行，也就是說，我們可以以一個陣列 $p[N]$ 來表示一組解。因為兩個皇后不可能在同一行，所以 $p[N]$ 必然是一個 $0 \sim N-1$ 的排列。我們可以嘗試所有可能的排列，對每一個排列來檢查是否有任兩個皇后在同一斜線上 (同行同列不需要檢查了)。

要產生所有排列通常是以字典順序 (lexicographic order) 的方式依序產生下一個排列，這裡我們不講這個方法，而介紹使用庫函數 `next_permutation()`，它的作用是找到目前排列在字典順序的下一個排列，用法是傳入目前排列所在的陣列位置 $[s, t)$ ，留意 C++ 庫函數中區間的表示方式幾乎都是左閉右開區間。回傳值是一個 `bool`，當找不到下一個的時候回傳 `false`，否則回傳 `true`。下面是迴圈窮舉的範例程式，其中檢查是否在同一條對角線可以很簡單的檢查 `abs(p[i]-p[j]) == j-i`。

```
#include <bits/stdc++.h>
using namespace std;

int nq(int n) {
```

列0	8			$p[0]=1$
列1			8	$p[1]=3$
列2		8		$p[2]=2$

29

$$\frac{|p[i]-p[j]|}{i-j} = 1 \quad \leftarrow j=i+1$$

$$p[i]-p[j] = i-j$$

$$= j-i$$

行1行2行3 用一維解二維,自帶行列只能出現一個

AP325 v:1.5 (20240921)

```
int p[14], total=0;
for (int i=0;i<n;i++) p[i]=i; //first permutation
do {
    // check valid
    bool valid=true;
    for (int i=0; i<n; i++) for (int j=i+1;j<n;j++)
        if (abs(p[i]-p[j])==j-i) { // one the same diagonal
            valid=false;
            break;
        }
    if (valid) total++;
} while (next_permutation(p, p+n)); // until no-next
return total;
}

int main() {
    for (int i=1;i<12;i++)
        printf("%d ",nq(i));
    return 0;
}
```

重新排列 p[]
直到出現 first permutation

接著看遞迴的寫法，每呼叫一次遞迴，我們檢查這一系列的每個位置是否可以放置皇后而不被之前所放置的皇后攻擊。對於每一個可以放的位置，我們就嘗試放下並往下一層遞迴呼叫，如果最後放到第 n 列就表示全部 n 列都放置成功。

```
// number of n-queens, recursion
#include<bits/stdc++.h>
using namespace std;

// k is current row, p[] are column indexes of previous rows
int nqr(int n, int k, int p[]) {
    if (k>=n) return 1; // no more rows, successful
    int total=0;
    for (int i=0;i<n;i++) { // try each column
        // check valid
        bool valid=true;
        for (int j=0;j<k;j++)
            if (p[j]==i || abs(i-p[j])==k-j) {
                valid=false;
                break;
            }
        if (!valid) continue;
        p[k]=i;
        total+=nqr(n,k+1,p);
    }
    return total;
}

int main() {
    int p[15];
    for (int i=1;i<12;i++)
        printf("%d ",nqr(i,0,p));
    return 0;
}
```

(3,0,p)


```
}

```

副程式中我們對每一個可能的位置 i 都以一個 j -迴圈檢查該位置是否被 $(j, p[j])$ 攻擊，這似乎有點缺乏效率。我們可以對每一個 $(j, p[j])$ 標記它在此列可以攻擊的位置，這樣就更有效率了。以下是這樣修改後的程式碼，這個程式大約比前一個快了一倍，比迴圈窮舉的方法則快了百倍。以迴圈窮舉的方法複雜度是 $O(n^2 \times n!)$ ，而遞迴的方法不會超過 $O(n \times n!)$ ，而且實際上比 $O(n \times n!)$ 快很多，因為你可以看得到，在遞迴過程中，被已放置皇后攻擊的位置都被略去了，所以實際上並未嘗試所有排列。這種情況下我們知道他的複雜度上限，但真正準確的複雜度往往難以分析，如果以實際程式來量測， $n=11$ 時， $n!=39,916,800$ ，但遞迴被呼叫的次數只有 16,6926 次，差了非常多。

```
// number of n-queens, recursion, better
#include<bits/stdc++.h>
using namespace std;

// k is current row, p[] are column indexes of previous rows
int nqr(int n, int k, int p[]) {
    if (k>=n) return 1; // no more rows, successful
    int total=0;
    bool valid[n];
    for (int i=0;i<n;i++) valid[i]=true;
    // mark positions attacked by (j,p[j])
    for (int j=0; j<k; j++) {
        valid[p[j]]=false; // 同列
        int i=k-j+p[j]; // p[j]+(k-j)
        if (i<n) valid[i]=false; // 斜率=+1, (k-j)是列之間的距離
        i=p[j]-(k-j);
        if (i>=0) valid[i]=false; // 斜率=-1
    }
    for (int i=0;i<n;i++) { // try each column
        if (valid[i]) {
            p[k]=i;
            total+=nqr(n,k+1,p);
        }
    }
    return total;
}

int main() {
    int p[15];
    for (int i=1;i<12;i++)
        printf("%d ",nqr(i,0,p));
    return 0;
}
```

以下是一個類似的習題。

Q-1-10. 最多得分的皇后

在一個 $n \times n$ 的方格棋盤上每一個格子都有一個正整數的得分，如果將一個皇后放在某格子上就可以得到該格子的分數，請問在放置的皇后不可以互相攻擊的條件下，最多可以得到幾分，皇后的個數不限制。 $0 < n < 11$ 。每格得分數不超過 100。

Time limit: 1 秒

輸入格式：第一行是 n ，接下來 n 行是格子分數，由上而下，由左而右，同行數字以空白間隔。

輸出格式：最大得分。

範例輸入：

```
3
1 4 2
5 3 2
7 8 5
```

範例輸出：

```
11
```

說明：選擇 4 與 7。

(請注意：是否限定恰好 n 個皇后答案不同，但解法類似，本題不限恰好 N 個皇后，所以遞迴搜尋過程要考慮某些列不放皇后的可能)

上面幾個例子都可以看到遞迴窮舉通常比迴圈窮舉來得有效率，有些問題迴圈的方法甚至很不好寫，而遞迴要容易得多，以下的習題是一個例子。

Q-1-11. 刪除矩形邊界 — 遞迴 (APCS201910, subtask)

一個矩形的邊界是指它的最上與最下列以及最左與最右行。對於一個元素皆為 0 與 1 的矩陣，每次可以刪除四條邊界的其中之一，要以逐步刪除邊界的方式將整個矩陣全部刪除。刪除一個邊界的成本就是「該邊界上 0 的個數與 1 的個數中較小的」。例如一個邊界如果包含 3 個 0 與 5 個 1，刪除該邊界的成本就是 $\min\{3, 5\} = 3$ 。

根據定義，只有一列或只有一行的矩陣的刪除成本是 0。不同的刪除順序會導致不同的成本，本題的目標是要找到最小成本的刪除順序。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 m 和 n ，以下 m 行是矩陣內容，順序是由上而下，由左至右，矩陣內容為 0 或 1，同一行數字中間以一個空白間隔。 $m + n \leq 13$ 。

輸出格式：最小刪除成本。

範例輸入：

```
3 5
0 0 0 1 0
1 0 1 1 1
0 0 0 1 0
```

範例輸出：

```
1
```

提示：將目前的矩陣範圍當作遞迴傳入的參數，對四個邊界的每一個，遞迴計算刪除該邊界後子矩陣的成本，對四種情形取最小值，遞迴的終止條件是：如果只有一列或一行則成本為 0。（本題有更有效率的 DP 解法）